

Домашнее задание

Storage Policy и резервное копирование в ClickHouse

Курс	ClickHouse
Среда выполнения	Виртуальная машина Yandex Cloud
Object Storage bucket	clickhouse-backups-otus
Сервисный аккаунт	clickhouse-backup-sa
Service account ID	aje0d114vb47jcd5t4fk
Static key ID	YCAJEiv2o68u9eTFRf3UgWlc
Secret key	скрыт в отчете
ClickHouse	25.8.22.28 official build
clickhouse-backup	2.7.1

1. Цель работы

Цель работы - изучить резервное копирование в ClickHouse: настроить storage policy, создать тестовую базу данных, выполнить backup на удаленный S3-совместимый ресурс, повредить данные и восстановить их из резервной копии.

2. Архитектура решения

Работа выполнялась на виртуальной машине в Yandex Cloud. ClickHouse Server, ClickHouse Client и clickhouse-backup были установлены на VM. В качестве удаленного S3-совместимого хранилища использовался Yandex Object Storage.

```
Yandex Cloud
├── Compute Cloud VM
│   ├── Ubuntu
│   ├── ClickHouse Server
│   ├── ClickHouse Client
│   └── clickhouse-backup
└── Object Storage
    └── bucket: clickhouse-backups-otus
```

3. Object Storage, сервисный аккаунт и ключи

Для резервных копий был создан bucket Object Storage `clickhouse-backups-otus`. Для доступа к bucket был создан сервисный аккаунт `clickhouse-backup-sa` и static access key. Секретный ключ в отчете не публикуется.

```
service_account_name: clickhouse-backup-sa
service_account_id: aje0d114vb47jcd5t4fk
access_key_id: YCAJEiv2o68u9eTFRf3UgWlc
secret_key: ***
bucket: clickhouse-backups-otus
endpoint: https://storage.yandexcloud.net
region: ru-central1
```

4. Настройка storage policy

Первоначально была попытка настроить S3-диск в storage policy. ClickHouse не запускался из-за ошибки доступа к S3-диску: `Access Denied, bucket clickhouse-backups`. Для продолжения выполнения задания была настроена локальная storage policy `local\_policy`, а удаленное резервное копирование выполнялось через clickhouse-backup в Yandex Object Storage.

```
sudo tee /etc/clickhouse-server/config.d/storage_policy.xml > /dev/null <<'EOF'
<clickhouse>
  <storage_configuration>
    <policies>
      <local_policy>
        <volumes>
          <main>
            <disk>default</disk>
          </main>
        </volumes>
      </local_policy>
    </policies>
  </storage_configuration>
</clickhouse>
EOF

sudo systemctl restart clickhouse-server
```

Проверка политики хранения выполнялась запросом к системной таблице:

```
SELECT
  policy_name,
  volume_name,
  volume_priority,
  disks
FROM system.storage_policies;
```

5. Установка clickhouse-backup

Утилита clickhouse-backup была скачана с GitHub и установлена из архива. Внутри архива бинарный файл находился по пути `build/linux/amd64/clickhouse-backup`.

```
cd /tmp
wget https://github.com/Altinity/clickhouse-backup/releases/download/v2.7.1/clickhouse-backup-linux-amd64.tar.gz
mkdir -p ch-backup
tar -xzf clickhouse-backup-linux-amd64.tar.gz -C ch-backup
sudo cp /tmp/ch-backup/build/linux/amd64/clickhouse-backup /usr/local/bin/clickhouse-backup
sudo chmod +x /usr/local/bin/clickhouse-backup
clickhouse-backup version
```

Проверка версии показала установленную версию `2.7.1`.

6. Конфигурация clickhouse-backup

Для отправки резервных копий в Yandex Object Storage был создан конфигурационный файл `/etc/clickhouse-backup/config.yml`.

```

general:
  remote_storage: s3
  backups_to_keep_local: 3
  backups_to_keep_remote: 5

clickhouse:
  username: default
  password: ""
  host: localhost
  port: 9000
  data_path: /var/lib/clickhouse
  skip_tables:
    - system.*

s3:
  access_key: YCAJEiv2o68u9eTFrRf3UgWlc
  secret_key: "****"
  bucket: clickhouse-backups-otus
  endpoint: https://storage.yandexcloud.net
  region: ru-central1
  path: clickhouse
  force_path_style: true
  disable_ssl: false
  acl: private

```

7. Создание тестовой базы и таблиц

Была создана база данных `backup\_test` и две таблицы: `users` и `orders`. Обе таблицы используют engine `MergeTree` и storage policy `local\_policy`.

```

CREATE DATABASE IF NOT EXISTS backup_test;

CREATE TABLE backup_test.users
(
  id UInt64,
  name String,
  email String,
  created_at DateTime
)
ENGINE = MergeTree
ORDER BY id
SETTINGS storage_policy = 'local_policy';

CREATE TABLE backup_test.orders
(
  order_id UInt64,
  user_id UInt64,
  amount Decimal(10, 2),
  status String,
  created_at DateTime
)
ENGINE = MergeTree
ORDER BY order_id
SETTINGS storage_policy = 'local_policy';

```

8. Заполнение таблиц и проверка исходных данных

```

INSERT INTO backup_test.users VALUES
  (1, 'Ivan Petrov', 'ivan@example.com', now()),
  (2, 'Anna Sidorova', 'anna@example.com', now()),
  (3, 'Petr Ivanov', 'petr@example.com', now());

INSERT INTO backup_test.orders VALUES
  (101, 1, 1500.50, 'paid', now()),
  (102, 2, 2300.00, 'new', now()),
  (103, 1, 999.99, 'cancelled', now());

```

Перед созданием backup были проверены данные и количество строк в таблицах.

```
SELECT *
FROM backup_test.users
ORDER BY id ASC
```

Query id: 02836d27-7d19-4918-9dd1-26b23b194cdb

	id	name	email	created_at
1.	1	Ivan Petrov	ivan@example.com	2026-06-16 08:23:23
2.	2	Anna Sidorova	anna@example.com	2026-06-16 08:23:23
3.	3	Petr Ivanov	petr@example.com	2026-06-16 08:23:23

3 rows in set. Elapsed: 0.002 sec.

```
SELECT *
FROM backup_test.orders
ORDER BY order_id ASC
```

Query id: cf060492-aa2b-414e-aca1-aa73035482b6

	order_id	user_id	amount	status	created_at
1.	101	1	1500.5	paid	2026-06-16 08:23:23
2.	102	2	2300	new	2026-06-16 08:23:23
3.	103	1	999.99	cancelled	2026-06-16 08:23:23

3 rows in set. Elapsed: 0.002 sec.

```
SELECT count()
FROM backup_test.users
```

Query id: a74edc24-cbb7-4c79-87bc-b667dd5a951b

	count()
1.	3

Рисунок 1 - Исходные данные в таблицах backup_test.users и backup_test.orders

```
SELECT count()
FROM backup_test.orders
```

Query id: 99a53190-e883-4c41-afba-dee6bd11d283

	count()
1.	3

1 row in set. Elapsed: 0.001 sec.

Рисунок 2 - Проверка количества строк в backup_test.orders

9. Создание и загрузка резервной копии

Команды `clickhouse-backup` выполнялись через `sudo`, так как каталог `/var/lib/clickhouse/backup` принадлежит пользователю ClickHouse.

```
sudo clickhouse-backup tables
sudo clickhouse-backup create backup_test_full
sudo clickhouse-backup upload backup_test_full
sudo clickhouse-backup list
```

В результате `backup\_test\_full` появился одновременно как локальная и удаленная резервная копия.

```

yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp$ sudo clickhouse-backup list
2026-06-16 08:31:15.211 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://localhost:9000
2026-06-16 08:31:15.212 INF pkg/clickhouse/clickhouse.go:1251 > SELECT value FROM `system`.`build_options` where name='VERSION_INTEGER'
2026-06-16 08:31:15.214 INF pkg/clickhouse/clickhouse.go:1251 > SELECT countIf(name='type') AS is_disk_type_present, countIf(name='object_storage_type') AS is_object_storage_type_present, countIf(name='free_space') AS is_free_space_present, countIf(name='disks') AS is_storage_policy_present, countIf(name='cache_path') AS is_cache_path_present FROM system.columns WHERE database='system' AND table IN ('disks','storage_policies')
2026-06-16 08:31:15.217 INF pkg/clickhouse/clickhouse.go:1251 > SELECT d.path AS path, argMin(d.name, d.cache_path != '') AS name, any(lower(if(d.type='ObjectStorage',d.object_storage_type,d.type))) AS type, min(d.free_space) AS free_space, groupUniqArray(s.policy_name) AS storage_policies FROM system.disks AS d LEFT JOIN (SELECT policy_name, arrayJoin(disks) AS disk FROM system.storage_policies) AS s ON s.disk = d.name GROUP BY d.path
2026-06-16 08:31:15.220 INF pkg/clickhouse/clickhouse.go:384 > clickhouse connection closed
2026-06-16 08:31:15.221 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://localhost:9000
2026-06-16 08:31:15.287 INF pkg/storage/general.go:292 > , list_duration=38.157242
2026-06-16 08:31:15.287 INF pkg/clickhouse/clickhouse.go:384 > clickhouse connection closed
backup_test_full 2026-06-16 08:30:48 local all:61.44MiB,data:61.37MiB,arch:0B,obj:0B,meta:69.78KiB,rbac:369B,conf:0B,nc:0B
regular
backup_test_full 2026-06-16 08:31:10 remote all:61.63MiB,data:61.37MiB,arch:61.61MiB,obj:0B,meta:13.65KiB,rbac:4.00KiB,conf:0B,nc:0B tar, regular

```

Рисунок 3 - Резервная копия backup_test_full создана локально и загружена в удаленное S3-хранилище

10. Повреждение данных

Для проверки восстановления данные были намеренно повреждены: таблица `orders` была удалена, а в таблице `users` значение имени для `id = 1` изменено на `BROKEN DATA`.

```
clickhouse-client --query "DROP TABLE backup_test.orders;"
```

```

clickhouse-client --query "
ALTER TABLE backup_test.users
UPDATE name = 'BROKEN DATA'
WHERE id = 1;
"

```

```

clickhouse-client --query "SHOW TABLES FROM backup_test;"
clickhouse-client --query "SELECT * FROM backup_test.users ORDER BY id;"

```

```

yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp$ clickhouse-client --query "DROP TABLE backup_test.orders;"

clickhouse-client --query "
ALTER TABLE backup_test.users
UPDATE name = 'BROKEN DATA'
WHERE id = 1;
"
yc-user@compute-vm-2-4-30-ssd-1776951461260:/tmp$ clickhouse-client --query "SHOW TABLES FROM backup_test;"

clickhouse-client --query "SELECT * FROM backup_test.users ORDER BY id;"
users
1      BROKEN DATA      ivan@example.com      2026-06-16 08:23:23
2      Anna Sidorova     anna@example.com      2026-06-16 08:23:23
3      Petr Ivanov       petr@example.com       2026-06-16 08:23:23

```

Рисунок 4 - Повреждение данных: таблица orders удалена, строка users изменена на BROKEN DATA

11. Восстановление данных

Первое восстановление завершилось ошибкой `Directory for table data store/... already exists`. Причина: база данных ClickHouse с engine Atomic оставила физические директории таблиц в `/var/lib/clickhouse/store` после удаления базы. Были найдены и удалены остаточные директории таблиц по UUID, после чего восстановление было выполнено повторно.

```

clickhouse-client --query "DROP DATABASE IF EXISTS backup_test SYNC;"

sudo systemctl stop clickhouse-server

sudo find /var/lib/clickhouse/store -type d \( -name "17edacbf-4379-4ddb-99aa-50b3d1a408d2" -o -name "68e0c8c2-c578-4585-a1d9-19896bcaedec" \) -print

sudo rm -rf /var/lib/clickhouse/store/17e/17edacbf-4379-4ddb-99aa-50b3d1a408d2
sudo rm -rf /var/lib/clickhouse/store/68e/68e0c8c2-c578-4585-a1d9-19896bcaedec

sudo systemctl start clickhouse-server
sudo clickhouse-backup restore -t 'backup_test.*' backup_test_full

```

Команда восстановления была ограничена только тестовой базой `backup\_test`, чтобы не затрагивать другие базы и таблицы на сервере.

12. Финальная проверка восстановления

После восстановления были выполнены запросы `SHOW TABLES`, `SELECT \*` и `count()` для обеих таблиц. Таблица `orders` снова появилась, в `users` значение `BROKEN DATA` было заменено исходным значением `Ivan Petrov`, количество строк в обеих таблицах равно 3.

```
yc-user@compute-vm-2-4-30-ssd-17/6951461260:/tmp$ clickhouse-client --query "SHOW TABLES FROM backup_test;"
clickhouse-client --query "SELECT * FROM backup_test.users ORDER BY id;"
clickhouse-client --query "SELECT * FROM backup_test.orders ORDER BY order_id;"
clickhouse-client --query "SELECT count() FROM backup_test.users;"
clickhouse-client --query "SELECT count() FROM backup_test.orders;"
orders
users
1      Ivan Petrov      ivan@example.com      2026-06-16 08:23:23
2      Anna Sidorova    anna@example.com      2026-06-16 08:23:23
3      Petr Ivanov      petr@example.com       2026-06-16 08:23:23
101    1      1500.5    paid    2026-06-16 08:23:23
102    2      2300    new     2026-06-16 08:23:23
103    1      999.99   cancelled 2026-06-16 08:23:23
3
3
```

Рисунок 5 - Финальная проверка: таблицы и данные успешно восстановлены из backup

13. Возникшие ошибки и решения

Ошибка 1 - Access Denied при настройке S3-диска

При настройке S3-диска в ClickHouse storage policy сервер не запускался с ошибкой доступа к bucket. Решение: для учебной работы была настроена `local\_policy`, а удаленное хранение backup реализовано через clickhouse-backup и Object Storage.

Ошибка 2 - clickhouse-backup: command not found

После скачивания архива бинарный файл не находился напрямую в `/tmp`, а лежал внутри архива по пути `build/linux/amd64/clickhouse-backup`. Решение: распаковать архив в отдельную папку и скопировать бинарный файл в `/usr/local/bin`.

Ошибка 3 - permission denied для /var/lib/clickhouse/backup

При запуске clickhouse-backup от пользователя `yc-user` возникла ошибка доступа к `/var/lib/clickhouse/backup`. Решение: выполнять команды `clickhouse-backup` через `sudo`.

Ошибка 4 - AccessDenied для bucket clickhouse-backups

При работе с первым bucket возникала ошибка `403 AccessDenied`. Решение: использовать bucket `clickhouse-backups-otus` и указать его в `/etc/clickhouse-backup/config.yml`.

Ошибка 5 - Directory for table data store already exists

При восстановлении после удаления базы ClickHouse не смог создать таблицы с теми же UUID, так как старые директории еще существовали в `/var/lib/clickhouse/store`. Решение: выполнить `DROP DATABASE IF EXISTS backup\_test SYNC`, остановить ClickHouse, удалить две остаточные директории UUID и повторить restore только для `backup\_test.\*`.

14. Вывод

В ходе работы была настроена виртуальная машина в Yandex Cloud, создан bucket Yandex Object Storage, установлен и настроен clickhouse-backup. Была создана тестовая база `backup\_test` с таблицами `users` и `orders`, выполнено резервное копирование в удаленное S3-совместимое хранилище, данные были намеренно повреждены и затем успешно восстановлены из резервной копии.

Результат восстановления подтвержден финальной проверкой: обе таблицы существуют, исходные данные восстановлены, количество строк в каждой таблице равно 3.